

DOCUMENTO DE DISEÑO DE JUEGO

Proyecto: Shooter Espacial 2D

Autor: Gabriel Samuel Vigil Rodríguez

Asignatura: Fundamentos de Videojuegos

Índice

Contenido

Índice	2
1. Escenas	3
Arquitectura de la escena.....	3
2. Assets y GameObjects.....	3
GameObjects	3
Prefabs	3
3. Mecánicas	3
Entorno limitado y desplazamiento.....	3
Generación de enemigos y Dificultad.....	4
Fragmentación de enemigos	4
4. Scripts	4
Relación de clases	4
5. Interacción	4

1.Escenas

Arquitectura de la escena

El juego se desarrolla íntegramente en una única escena principal (SampleScene). Al ser un proyecto introductorio a la materia y para mantenerlo lo más simple posible no se han necesitado más. Se entiende que esto elimina completamente tiempos de carga entre menús y el núcleo jugable.

El ciclo de vida de la partida, incluyendo los estados de inicio, pausa, pérdida y reinicio, se controlan mediante la manipulación de la escala de tiempo del motor físico (Time.timeScale). Un lienzo superpuesto (Canvas) gestiona la actualización de la puntuación en tiempo real y el menú de interrupción de la partida.

2.Assets y GameObjects

GameObjects

- Main Camera: Cámara ortográfica configurada para la visión 2D. Inicializa y gestiona el script de MeteoriteSpawner, encargado de gestionar la instanciación de los enemigos, en este caso meteoritos, en el límite superior del espacio.
- Player: Nave espacial contralada por el jugador. Contiene componente Rigidbody y Collider para la interacción con los obstáculos.
- Pool: entidad que contiene y gestiona el script de BulletPool.
- GameManager: objeto que aloja el script PauseMenu y procesa los eventos de la interfaz de usuario.
- Canvas: contiene la interfaz gráfica que incluye el texto con la puntuación y los botones de Reanudar y Reiniciar.

Prefabs

Se han usado 3 prefabs para llevar a cabo este proyecto:

- Bullet: prefab de las balas de la nave. Tiene un sistema de detección de colisiones basado en triggers para interactuar con los objetivos.
- Meteor: prefab de los enemigos grandes del juego. Desciende a una velocidad constante, la gravedad, y alberga la lógica para instanciar sus versiones más pequeñas.
- Fragment: prefab generado en tiempo de ejecución al destruir un meteorito grande.

3.Mecánicas

Entorno limitado y desplazamiento

El jugador se desplaza mediante un sistema de físicas impulsado por Rigidbody.AddForce, lo que simula la inercia espacial. Para garantizar que la nave se mantenga dentro del encuadre de la cámara se ha programado una mecánica de teletransporte: cuando las coordenadas de la nave exceden los límites predefinidos (xBorderLimit, yBorderLimit), esta reaparece instantáneamente en el extremo opuesto.

Generación de enemigos y Dificultad

El sistema de MeteoriteSpawner introduce enemigos en el eje Y con coordenadas X aleatorias. Además, integra una mecánica de dificultad escalable: la tasa de aparición (ratePerMinute) se incrementa conforme avanza el tiempo que pasa, incrementando la dificultad general del juego.

Fragmentación de enemigos

Al registrar la colisión de una bala, el meteorito es destruido y ejecuta un método de fragmentación. Inmediatamente antes de su liberación de memoria, instancia 2 copias del prefab Fragment en sus coordenadas exactas. A estas nuevas entidades se les otorgan vectores de dirección diagonales opuestos, requiriendo que se elimine cada uno de esos fragmentos por separado.

4. Scripts

Relación de clases

- Player.cs: script relacionado con la nave. Calcula las físicas de inercia, el reposicionamiento en bordes, la detección de colisiones y almacena la puntuación.
- Enemy.cs, Fragment.cs y MeteoriteSpawner: se encargan de los comportamientos de los meteoritos como de los fragmentos, tanto sus movimientos, sus colisiones y la destrucción de estos.
- Bullet.cs y BulletPool.cs: se encargan de la generación y el comportamiento de las balas. Bullet.cs encargándose de la velocidad y tiempo de vida, además de manejar una flag enUso utilizada por BulletPool.cs. Para la generación de balas se ha usado una estrategia de pooling. Cuando una bala choca con un enemigo o excede su tiempo de vida cambia su estado de enUso a false y desplaza sus coordenadas fuera del campo visible del juego, quedando preparado para ser invocado y reposicionado en la siguiente orden de disparo del jugador.
- PauseMenu.cs: actúa como puente de comunicación entre la interfaz de usuario y el bucle principal de Unity. Detiene y reanuda el tiempo alterando Time.timeScale y restaura el estado inicial mediante SceneManager.

5. Interacción

El sistema de entrada mapea las acciones que realiza la nave a teclas estándar:

- Movimiento de la nave: la nave es propulsada hacia delante o hacia atrás mediante las teclas W/S respectivamente, o con las Flechas Arriba/Abajo. Esto aplica una fuerza física al componente Rigidbody que provoca una sensación de peso en la nave.
- Rotación de la nave: el giro de la nave de izquierda a derecha es controlado por las teclas A/D respectivamente, o con las Flechas Izquierda/Derecha. Esto modifica el ángulo de giro sobre el eje Z para apuntar.
- Disparo de la nave: con la tecla ESPACIO. Extrae un proyectil disponible del almacén de memoria.
- Control de la partida: con la tecla ESC/P. Detiene el motor de físicas y despliega el menú de interfaz.